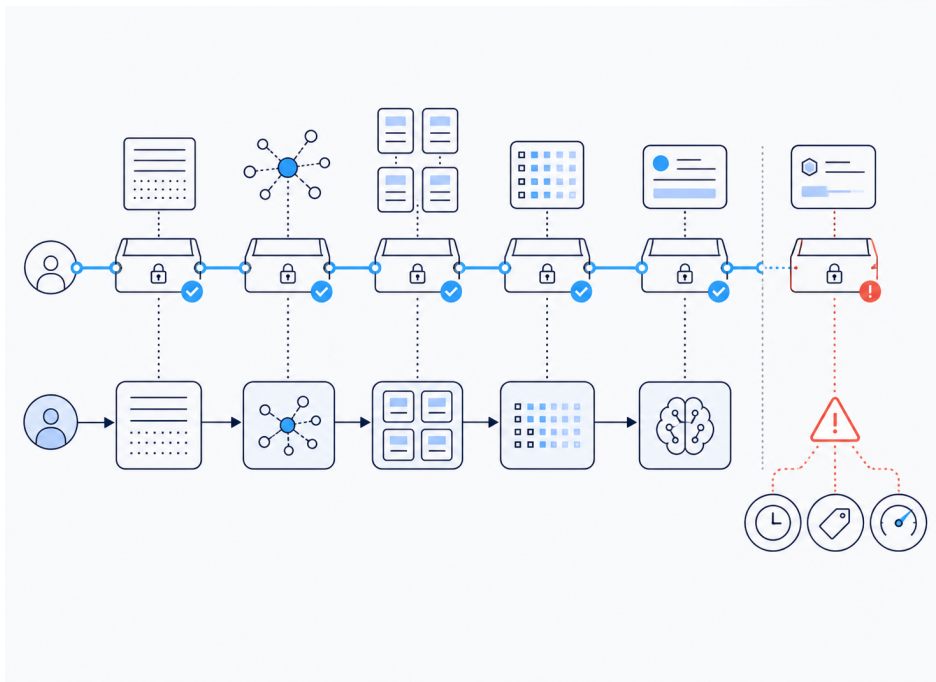


Caching For RAG & Agents



People hear "caching" and think prompt caching. A RAG agent has **six places to cache**, and the cheapest one to add is the most dangerous to trust.



The Biggest Cost Lever, The Easiest To Get Wrong

PROBLEM

A cache hit is 100x cheaper. A wrong cache hit is a confident wrong answer with no error and no log.

🔴 UNDER-CACHING

a cache miss on a repeat
you pay full cost + latency
visible on a dashboard
a slow bill

🔴 OVER-CACHING

a hit on the wrong query
instant, free, and WRONG
invisible until a complaint
a silent lie



Every other optimisation fails loudly. A bad cache hit returns a **plausible, well-formed answer that happens to be wrong** — instantly, with no call to inspect and no error to catch.



AI Coach John

FOLLOW



REPOST

Exact And Semantic Caching Are Not The Same Tool

DISTINCTION

One matches bytes and is always safe. One matches meaning and is never quite safe.

	EXACT CACHE	SEMANTIC CACHE
Keys on	the literal bytes	meaning (embedding)
Hits when	input is identical	similarity > threshold
Correct	always	never certain
Misses	on any variation	rarely
Danger	none	wrong-but-similar hit



Use **exact caching** wherever the input can repeat (prefix, normalised query, tool args). Reserve **semantic caching** for cases where a wrong hit is cheap. They are complements, not alternatives.



The Naive Cache That Ships A Wrong Answer

EXAMPLE

Cache the response, key on meaning, threshold 0.95. Works in the demo, lies in production.

THE CACHE ALMOST EVERYONE ADDS FIRST

```
q_emb = embed(query)
hit = cache.search(q_emb, threshold=0.95) # cosine
if hit:
    return hit.response # instant, free
answer = rag_pipeline(query)
cache.store(q_emb, answer)
```

STORED

"return a laptop after 30 days?"

answer: "No, the limit is 30 days."

cosine 0.97 ≥ threshold 0.95 → CACHE HIT

ASKED

"return a laptop after 20 days?"

correct answer is "Yes" — the cache serves "No"

the fact lives in the number the embedding treated as noise



Prompt Prefix Caching

EXACT · ALWAYS SAFE

The safest, biggest, first win. Cache the stable front of every request: system prompt and tool definitions.

STABLE – identical every call

system prompt
tool definitions

VOLATILE – different every call

retrieved chunks
the user query

cache breakpoint

one changing byte in the stable region invalidates all of it



Cuts the cost of those tokens by ~90% and removes their processing latency on every request. Exact, always correct, pure upside. **Add this first.**

↑↑ THE ONE RULE

Put timestamps and per-request ids **after** the cache breakpoint. A single moving byte near the front, or an unsorted tool list, kills the cache for everything after it.



AI Coach John

FOLLOW



REPOST

The Query Embedding Cache

EXACT · ALWAYS SAFE

Embedding the query is a network call on every request. The same question embeds to the same vector, so cache it.



NORMALISE, THEN CACHE

VARIANT 1

"Can I return a laptop?" → normalise

VARIANT 2

"can i return a laptop" → normalise

SAME KEY

"can i return a laptop" → exact hit, skip the embed call

- ✓ Lowercase, trim whitespace, collapse punctuation before the key. This turns **formatting noise into safe exact hits**, with zero semantic risk — you are not guessing at meaning.



A small, safe, unglamorous win that teams skip because they are focused on the **model call downstream**. A handful of phrasings cover most traffic.



AI Coach John

FOLLOW

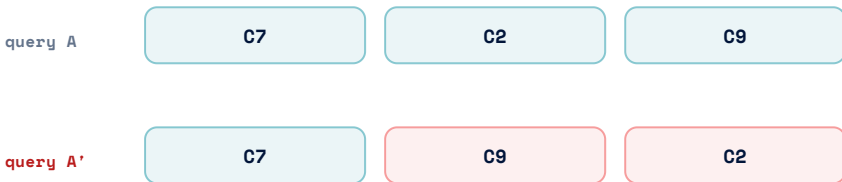


REPOST

Retrieval Cache, And Why RAG Breaks Prefix Caching

THE TRAP

Tiny query changes reshuffle the chunks, which is exactly why provider prefix caching barely helps RAG.



one reordered chunk breaks the byte-exact prefix for everything after it



Teams enable prompt caching, expect a big RAG saving, and get **almost nothing**: the retrieved content changes too often to cache as a prefix.

↑↑ THE FIX

Keep stable content (system, tools) **before** any retrieved content. Cache retrieval results **separately, keyed on the query**, rather than hoping the assembled prompt repeats.



AI Coach John

FOLLOW



REPOST

KV-Cache Reuse Of Retrieved Chunks

THE FRONTIER

The same popular chunk gets reprocessed in thousands of requests. Advanced RAG caches the model's work on it, not just the text.

WITHOUT

```
chunk text
  ↓
model recomputes
KV tensors
  ↓
every request
```

WITH KV REUSE

```
chunk text
  ↓
[ precomputed
  KV tensors ]
  ↓
spliced in
```

-  The return-policy chunk appears in most requests. Caching its **text** saves retrieval; caching its **tensors** saves the model's work. A small set of chunks dominates retrieval.



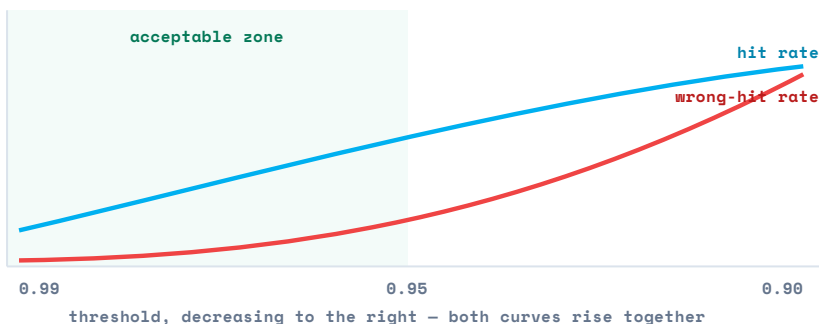
Not a config flag: it needs an inference stack that supports chunk-level KV reuse, and reused tensors are **position-dependent**, so naive reuse can change outputs.



The Response Cache, And The Threshold

THE WHOLE GAME

Caching the final answer is the biggest latency win and the biggest risk. The similarity threshold decides how often you lie.



DEFINITIONS

phrasing-insensitive → a low threshold is fine

POLICY / PRICE

a single word flips the answer → high, or exact only



There is **no universal correct threshold**. It depends on how much your answers change with small wording changes. Set it **per question type**, not globally.



AI Coach John

FOLLOW



REPOST

Cache Invalidation Is The Hard Problem, **Semantic** Makes It Harder

SILENT FAILURE

The knowledge base changed. Every cached answer from the old version is now confidently stale.

🔍 EXACT + PROVENANCE

the answer is tagged "from doc 47"
→ delete every entry for doc 47.

Precise and complete.

? SEMANTIC, NO PROVENANCE

the answer is keyed on a query embedding, with no link back to doc 47. **You can only TTL-expire and hope.**

🛡️ THREE DEFENCES

1

TTL on every answer. Bounds staleness by time even when you cannot target it.

2

Provenance tagging. Store which documents an answer came from, so exact invalidation is possible.

3

Event-driven flush. Clear the relevant cache region when a source document changes.



Caching For Agents: Tool Results And Reasoning

AGENTS ONLY

An agent repeats work a chatbot never does. It calls the same tool with the same args, across turns and sessions.

TOOL CALL	SAFE TO CACHE?	TTL
<code>lookup_policy("returns")</code>	yes, exact	hours
<code>get_product_specs(sku)</code>	yes, exact	hours
<code>get_order_status(id)</code>	only briefly	seconds
<code>check_live_inventory(sku)</code>	no	none



Tool-result caching is exact on the arguments — **no semantic risk**. Calling `lookup_policy("returns")` in a thousand sessions returns the same result.



Cache **by how fast the underlying data changes**, not with one global TTL. Reasoning and sub-agent caching is semantic and higher risk — reserve it for expensive, stable sub-tasks.



AI Coach John

FOLLOW



REPOST

Six Layers, Safe By Construction

FULL STACK

The exact layers carry the load. The semantic layer is small and fenced. Add them in this order.

LAYER	KEY	RISK	ADD IT
prompt prefix	exact bytes	none	first
query embedding	normalised text	none	second
tool results	the arguments	none*	second
retrieval results	normalised query	low	third
KV chunk reuse	the chunk	low	if supported
response (exact)	normalised query	none	fourth
response (semantic)	embedding + thr	high	last, fenced

Almost all the benefit comes from the **exact layers, which are always correct**. The risky semantic response cache is added last, per-type, with a TTL and provenance.

Hit Rate And Wrong-Hit Rate, Measured Together

[PROVE IT](#)

A 90% hit rate that is 3% wrong is a machine for producing confident errors.

MEASURE BOTH ON LABELLED QUERY PAIRS

```
def cache_quality(cache, eval_pairs):
    hits = wrong = total = 0
    for query, gold in eval_pairs:
        total += 1
        hit = cache.search(embed(query), threshold=T)
        if hit:
            hits += 1
            if not answers_match(hit.response, gold):
                wrong += 1 # a confident wrong hit
    return hits/total, wrong/max(hits,1) # sweep T, plot both
```



Optimising hit rate alone drives you to **lower the threshold, which raises wrong hits**. Pick the lowest threshold where the wrong-hit rate is still acceptable for that type — for policy, acceptable is zero.



AI Coach John

[FOLLOW](#)



[REPOST](#)

What Breaks After Launch

OPERATIONS

The cache was correct for the data you shipped. Both keep changing, and the cache does not notice.

CHANGE THAT LOOKS SAFE	WHAT IT BROKE
edited the return policy	cached answers now stale
added a timestamp to the prompt	prefix cache hit rate → 0
a new product launched	hit rate collapses, wrong keys
added a policy-sensitive doc	the safe threshold now lies

🔗 Monitor hit rate **and** wrong-hit rate as first-class metrics, invalidate on content change with provenance, and re-check that the prefix cache actually hits after every prompt change.

The Cost And Latency Win, Quantified

THE PAYOFF

On a repeat-heavy workload, caching is the difference between the bill you can afford and the one you cannot.

SAVING ON A CACHE HIT (typical / reported)



response hit skips retrieval AND the model call entirely

📈 The number that decides everything is your **repeat rate**. A high-repeat FAQ agent: huge upside. A novel-research agent: little to cache. **Measure it before you invest.**



AI Coach John

FOLLOW



REPOST

Exact Or Semantic, In One Table

KEY DECISION

For each thing you might cache, one question: can the input actually repeat exactly?

WHAT TO CACHE	CAN INPUT REPEAT EXACTLY?	KEY IT
system + tools	yes, every request	EXACT (prefix)
query text	yes, after normalising	EXACT
tool call + args	yes, same args	EXACT (+TTL)
retrieval results	yes, normalised query	EXACT
answer, factual	no, phrasing varies	SEMANTIC, careful
answer, policy	no, a word flips it	EXACT ONLY

🔄 Default to EXACT. Use SEMANTIC only when input never repeats AND a wrong hit is cheap. **Never semantic-cache policy or pricing answers.**

Setting The Semantic Threshold, From Your Own Data

PROCEDURE

Do not pick 0.95 because a blog said so. Derive it from your near-miss pairs.

1 Collect similar-but-different pairs. "return laptop after 20 days" vs "...30 days"; "pro plan price" vs "pro plus plan price".

2 Measure each dangerous pair's similarity. Most cluster around, say, 0.94 to 0.97.

3 Set the threshold ABOVE the worst near-miss. If the worst is 0.97, a 0.95 threshold lies. You need > 0.97 , or exact caching.

4 Re-derive when you add documents that create new near-misses.



The threshold is a **property of your data, not a magic constant**. A number that ignores your near-misses is a number that produces wrong hits.



Which Cache Should You Add?

DECISION PATH

Add caches in order of safety, and stop when your cost and latency targets are met.

Q1 **Is your workload repeat-heavy?** No → there is little on offer. Add prompt prefix caching for the input saving, and stop.

Q2 **Add the safe layers — all exact, zero risk. Targets met?** Yes → **stop**. prefix → query embed → tool results → retrieval. You got the wins with no risk.

Q3 **Still not enough after the exact response cache?** Then a semantic response cache — **only if** answers are insensitive to wording, a wrong hit is cheap, and you have a data-derived threshold + TTL + provenance.



Policy, pricing and eligibility answers stay **EXACT, forever**.



AI Coach John

FOLLOW



REPOST

What You See, Which Cache Decision Is Behind It

DEBUG TABLE

Most rows trace back to a semantic threshold or a missing invalidation.

SYMPTOM	CAUSE	LAYER
Wrong answer, instant, no error	semantic hit on a close query	threshold
Enabled prompt caching, saved little	chunks break the prefix	ordering
Cache hit rate dropped to 0	a byte changed in the prefix	prefix
Answers stale after an edit	no invalidation on change	provenance
Right usually, wrong on edge cases	threshold too low for the type	threshold
Agent slow despite a cache	tool results not cached	tool cache
Huge cache, low hit rate	caching novel, non-repeat questions	repeat rate
Fixed one wrong hit, another appeared	global threshold, mixed types	per-type



Cache The Byte, Fear The Meaning

CHEATSHEET

Exact caching is free correctness. Semantic caching quietly trades correctness for speed.

LAYER	THE ONE LINE THAT MATTERS
prompt prefix	stable content first, or the cache never hits
query embedding	normalise the text, then exact-cache it
tool results	exact on the arguments, TTL by data freshness
retrieval results	cache on the query, not the assembled prompt
KV chunk reuse	the frontier layer; take it when your stack has it
response (exact)	safe — add it before you ever go semantic
response (semantic)	last, fenced, per-type threshold, TTL, provenance
invalidation	provenance, or you only get a TTL
measure	hit rate AND wrong-hit rate, or you fly blind

 Exact caching is free correctness. **Semantic caching is a loan.**

